Implementing the E-Commerce System with SQL

From ER Diagram to SQL Database Implementation

Introduction

After designing the **ER Diagram**, the next step is to convert the conceptual design into an actual database using SQL.

This process is known as **ER-to-Relational Mapping**.

In this lesson, we will implement the E-Commerce database designed in the ER Diagram using SQL.

By the end of this implementation, the database will support:

- Customer Registration
- Product Categorization
- Shopping Cart Management
- Order Placement
- Payment Processing
- Product Reviews

Step 1: Creating the Database

```
CREATE DATABASE ecommerce_system;

USE ecommerce_system;
```

Step 2: Mapping ER Diagram to SQL Tables

ER Diagram Summary

Entity	SQL Table
Customer	Customers
Category	Categories
Product	Products
Cart	Cart
Orders	Orders

Entity	SQL Table
Order_Items	Order_Items
Payment	Payments
Review	Reviews

Step 3: Implementing Customer Entity

ER Representation

Customer

customer_id (PK)

name

email

phone

address

created_at

SQL Table

```
CREATE TABLE Customers(

customer_id INT AUTO_INCREMENT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

email VARCHAR(100) UNIQUE NOT NULL,

phone VARCHAR(15),

address TEXT,

created_at TIMESTAMP

DEFAULT CURRENT_TIMESTAMP

);
```

Relationship

Customer has many Orders

Customer has many Cart Items

Customer writes many Reviews

Step 4: Implementing Category Entity

ER Representation

Category

category_id (PK)

category_name

SQL Table

CREATE TABLE Categories(

category_id INT AUTO_INCREMENT PRIMARY KEY,

category_name VARCHAR(100)

NOT NULL UNIQUE

);

Step 5: Implementing Product Entity

ER Representation

Product

product_id (PK)

product_name

description

price

stock_quantity

category_id (FK)

SQL Table

CREATE TABLE Products(

product_id INT AUTO_INCREMENT PRIMARY KEY,

product_name VARCHAR(150)

NOT NULL,

description TEXT,

price DECIMAL(10,2)

NOT NULL,

stock_quantity INT

DEFAULT 0,

category_id INT,

FOREIGN KEY(category_id)

REFERENCES Categories(category_id)

);

Relationship Mapping

Category (1)

⬇

Products (Many)

One category contains many products.

Step 6: Implementing Cart Entity

ER Representation

Cart

cart_id (PK)

customer_id (FK)

product_id (FK)

quantity

SQL Table

CREATE TABLE Cart(

cart_id INT AUTO_INCREMENT PRIMARY KEY,

customer_id INT,

product_id INT,

quantity INT

DEFAULT 1,

FOREIGN KEY(customer_id)

REFERENCES Customers(customer_id),

FOREIGN KEY(product_id)

REFERENCES Products(product_id)

);

Relationship

Customer

1

⬇

Many

Cart

Cart

Many

⬇

1

Product

Step 7: Implementing Orders Entity

ER Representation

Orders

order_id (PK)

customer_id (FK)

order_date

status

SQL Table

CREATE TABLE Orders(

order_id INT AUTO_INCREMENT PRIMARY KEY,

customer_id INT,

order_date TIMESTAMP

DEFAULT CURRENT_TIMESTAMP,

status VARCHAR(50)

DEFAULT 'Pending',

FOREIGN KEY(customer_id)

REFERENCES Customers(customer_id)

);

Relationship

Customer

1

⬇

Many

Orders

Step 8: Implementing Order_Items Entity

Order_Items resolves the many-to-many relationship between Orders and Products.

ER Representation

Order_Items

order_item_id (PK)

order_id (FK)

product_id (FK)

quantity

price

SQL Table

```
CREATE TABLE Order_Items(

order_item_id INT

AUTO_INCREMENT PRIMARY KEY,

order_id INT,

product_id INT,

quantity INT,

price DECIMAL(10,2),

FOREIGN KEY(order_id)

REFERENCES Orders(order_id),

FOREIGN KEY(product_id)
```

REFERENCES Products(product_id)

);

Relationship

Orders

1

⬇

Many

Order_Items

Many

⬇

1

Products

Step 9: Implementing Payments Entity

ER Representation

Payments

payment_id (PK)

order_id (FK)

payment_method

amount

payment_status

payment_date

SQL Table

```
CREATE TABLE Payments(

payment_id INT

AUTO_INCREMENT PRIMARY KEY,

order_id INT UNIQUE,

payment_method

VARCHAR(50),

amount DECIMAL(10,2),

payment_status

VARCHAR(30),

payment_date

TIMESTAMP

DEFAULT CURRENT_TIMESTAMP,

FOREIGN KEY(order_id)

REFERENCES Orders(order_id)

);
```

Relationship

Orders

1

⬇

1

Payments

Step 10: Implementing Reviews Entity

ER Representation

Review

review_id (PK)

customer_id (FK)

product_id (FK)

rating

comment

SQL Table

CREATE TABLE Reviews(

review_id INT

AUTO_INCREMENT PRIMARY KEY,

customer_id INT,

product_id INT,

```
rating INT

CHECK(rating BETWEEN 1 AND 5),

comment TEXT,

FOREIGN KEY(customer_id)

REFERENCES Customers(customer_id),

FOREIGN KEY(product_id)

REFERENCES Products(product_id)

);
```

Step 11: Populating the Database

Insert Categories

```
INSERT INTO Categories(category_name)

VALUES

('Electronics'),

('Fashion'),

('Books');
```

Insert Customers

```
INSERT INTO Customers(

name,

email,
```

```
phone,
address
)
VALUES

('John Doe',
'john@gmail.com',
'9876543210',
'New York'),

('Jane Smith',
'jane@gmail.com',
'9988776655',
'California');
```

Insert Products

```
INSERT INTO Products(
product_name,
description,
price,
stock_quantity,
category_id
)
```

```
VALUES

('Laptop',
'Gaming Laptop',
75000,
15,
1),

('T-Shirt',
'Cotton T-Shirt',
800,
50,
2);
```

Step 12: Adding Products to Cart

```
INSERT INTO Cart(
customer_id,
product_id,
quantity
)
VALUES

(1,1,2),
(1,2,1);
```

Step 13: Creating an Order

Create Order

```
INSERT INTO Orders(customer_id)

VALUES(1);
```

Add Order Items

```
INSERT INTO Order_Items(

order_id,

product_id,

quantity,

price

)

VALUES


(1,1,2,75000),

(1,2,1,800);
```

Payment

```
INSERT INTO Payments(

order_id,

payment_method,

amount,

payment_status

)

VALUES
```

(1,

'UPI',

150800,

'Completed');

Step 14: Updating Product Stock

UPDATE Products

SET stock_quantity =

stock_quantity - 2

WHERE product_id=1;

Step 15: Removing Items from Cart

DELETE FROM Cart

WHERE customer_id=1;

Step 16: Querying the Database

Display Products

SELECT *

FROM Products;

Products with Category Name

SELECT

product_name,

category_name

FROM Products

JOIN Categories

ON Products.category_id

=

Categories.category_id;

Customer Cart Details

SELECT

Customers.name,

Products.product_name,

Cart.quantity,

Products.price,

(Cart.quantity*Products.price)

AS Total

FROM Cart

```
JOIN Customers

ON Customers.customer_id

= Cart.customer_id


JOIN Products

ON Products.product_id

= Cart.product_id;
```

Customer Order History

```
SELECT

Customers.name,

Orders.order_id,

Orders.status,

Orders.order_date

FROM Orders

JOIN Customers

ON Customers.customer_id
```

= Orders.customer_id;

Complete Order Details

```
SELECT

Orders.order_id,

Customers.name,

Products.product_name,

Order_Items.quantity,

Order_Items.price,

(Order_Items.quantity *

Order_Items.price)

AS Amount

FROM Order_Items

JOIN Orders

ON Orders.order_id

= Order_Items.order_id
```

JOIN Customers

ON Customers.customer_id

= Orders.customer_id

JOIN Products

ON Products.product_id

= Order_Items.product_id;

Product Reviews

SELECT

Customers.name,

Products.product_name,

Reviews.rating,

Reviews.comment

FROM Reviews

JOIN Customers

ON Customers.customer_id

= Reviews.customer_id

JOIN Products

ON Products.product_id

= Reviews.product_id;

Step 17: Using Transactions

Transactions ensure that all operations complete successfully.

START TRANSACTION;

INSERT INTO Orders(customer_id)

VALUES(1);

UPDATE Products

SET stock_quantity =

stock_quantity-1

WHERE product_id=1;

INSERT INTO Payments(

order_id,

payment_method,

amount,

payment_status

)

VALUES(

1,

'Credit Card',

75000,

'Completed'

);

COMMIT;

Rollback Example

ROLLBACK;

Step 18: Final Database Schema

Customers
   |
   ├──────< Orders
   | |
   | └──────< Order_Items >────── Products
   |
   ├──────< Cart >────── Products
   |
   └──────< Reviews >────── Products

Categories
   |
   └──────< Products

Orders

```
|
└──────── Payments
```

Best Practices

Use Constraints

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- CHECK
- NOT NULL

Use Indexes

```
CREATE INDEX idx_product_name

ON Products(product_name);


CREATE INDEX idx_customer_email

ON Customers(email);
```

Secure Customer Credentials

Store passwords in a separate column and save only hashed passwords.

Example:

$2b$12$K8M2a9nQ...

Conclusion

By converting the ER diagram into SQL tables, we successfully implemented a complete **E-Commerce Management System** consisting of **8 entities, 9 relationships, primary keys, foreign keys, constraints, transactions, and SQL queries**. This implementation demonstrates how conceptual database modeling is transformed into a fully functional relational database suitable for real-world e-commerce applications.